

Aegis DLP - Event Bus & Correlation Engine

This document provides a comprehensive, technically accurate explanation of the Event Bus and Correlation Engine implementation in the Aegis DLP project. The explanation is derived entirely from the underlying source code in `modules/event_bus/`.

1. High-Level Architecture

The Event Bus acts as the central nervous system of the Aegis DLP platform. Instead of having modules (like the Network IDS, File Monitor, and Phishing Detector) talk directly to each other, they use a

Publish/Subscribe (Pub/Sub) message broker pattern.

The lifecycle of an event looks like this:

1. **Event Generation:** A module detects something (e.g., a file is modified) and constructs a `SecurityEvent`.
 2. **Publishing:** The module publishes this event to the `EventBus`.
 3. **Distribution:** The `EventBus`:
 - Adds the event to an in-memory history ring buffer.
 - Saves it to an SQLite database via the `EventLogger`.
 - Broadcasts it to the frontend via `Socket.IO` for real-time dashboard updates.
 - Forwards it to the `CorrelationEngine` to check against threat rules.
 - Dispatches it asynchronously to any other modules that have explicitly subscribed to that event pattern.
-

2. The `SecurityEvent` Structure (`events.py`)

Everything revolves around the `SecurityEvent` dataclass. Every event flowing through the system has a strict schema:

- **event_id:** An auto-generated unique UUID.
- **event_type:** What happened (e.g., `EventType.FILE_MODIFIED`, `EventType.NET_ANOMALY`).
- **source:** Which module produced it (e.g., `ModuleSource.FILE_MONITOR`).
- **severity:** The threat level (`INFO`, `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`).
- **timestamp:** UTC time of occurrence.
- **data:** A dictionary containing module-specific payload details (like `file_hash`, `anomaly_score`, `process_id`).
- **correlation_id:** Used to link multiple events together if they trigger a compound rule.

To make generating events easier and consistent, the system uses factory helper functions like `file_event()`, `network_event()`, and `phishing_event()`.

3. The `EventBus` Class (`bus.py`)

The `EventBus` is a thread-safe singleton.

Subscriptions

Modules can subscribe to events using wildcard patterns. For example:

```
bus.subscribe(subscriber_id="my_module", pattern="file.*", handler=my_function)
```

This would map the `my_function` callback to execute whenever an event starting with `file.` is fired.

Subscriptions also support filtering by minimum severity or source module.

Async Dispatching

When an event is published via `bus.publish(event)`, the dispatch happens asynchronously. The bus places the event in a thread-safe `queue.Queue()`, and a dedicated background daemon thread (`EventBus-Dispatcher`) pulls events from the queue and executes the subscribed callbacks.

Fault Tolerance: If a subscriber's callback function crashes or raises an exception, the Event Bus catches the error, increments an `error_count` for that subscriber, and continues. If a subscriber fails 10 times consecutively (`MAX_SUBSCRIPTION_ERRORS = 10`), the Event Bus will auto-disable that specific subscription to protect the stability of the platform.

4. The CorrelationEngine (correlation.py)

While the Event Bus routes individual events, the `CorrelationEngine` looks for **compound threats** occurring across multiple modules over time.

It receives every event from the Event Bus via the `process_event()` method and stores them in a fast, time-windowed lookup buffer (`_event_buffer`), neatly organized by event type.

It evaluates these recent events against **Correlation Rules**. A rule matches when all of its trigger conditions are met within its designated `time_window`. If an event meets the criteria, it respects a `cooldown` period to prevent alert spamming.

The Engine currently enforces 8 hardcoded threat detection rules:

1. **PHISHING_MALWARE_CHAIN** (Critical): Detects a `PHISH_DETECTED` event and a `MALWARE_DETECTED` event within 1 hour.
2. **UNAUTHORIZED_DATA_THEFT** (Critical): Detects an unauthorized USB (`USB_UNAUTHORIZED`) and a sensitive file scan (`DATA_SENSITIVE`) within 5 minutes.
3. **RANSOMWARE_PATTERN** (Critical): A count-based rule. Detects 50+ `FILE_MODIFIED` events within 60 seconds.
4. **USB_NETWORK_CORRELATION** (High): Detects a USB insertion (`USB_INSERTED`) and a network anomaly (`NET_ANOMALY`) within 5 minutes.
5. **MALWARE_C2_DETECTED** (Critical): Detects malware (`MALWARE_DETECTED`) and anomalous network traffic (`NET_ANOMALY`) within 10 minutes, assuming Command & Control (C2) communication.
6. **BULK_SENSITIVE_ACCESS** (High): A count-based rule. Detects 10+ sensitive data accesses (`DATA_SENSITIVE`) within 5 minutes.

7. **FAILED_AUTH_BRUTE_FORCE** (High): A count-based rule. Detects 5+ authorization failures (**USB_UNAUTHORIZED**) within 2 minutes.
8. **PHISHING_FILE_EXEC** (High): Detects a phishing email (**PHISH_DETECTED**) followed by a new file creation (**FILE_CREATED**) within 30 minutes.

When a rule triggers, it generates a unique **correlation_id**, assigns it to all matched events, sends a special **correlation_alert** via Socket.IO, and passes the associated response actions to the **ResponseExecutor**.

5. The **ResponseExecutor** (**responses.py**)

The **ResponseExecutor** is responsible for taking automated action when the Correlation Engine detects a threat. Action names defined in the rules (like **"block_usb"** or **"quarantine_file"**) are mapped to registered Python functions.

It contains 14 default response actions, including:

- **_handle_send_alert / _handle_send_critical_alert / _handle_send_emergency_alert**: Publishes system-wide alert events back to the Event Bus.
- **_handle_block_usb**: Publishes a **SYSTEM_CORRELATION_MATCH** event addressed to the **usb_monitor** module with instructions to block transfers.
- **_handle_quarantine_file**: Reads the file paths from the matching events and prepares them for isolation.
- **_handle_kill_process**: Identifies PIDs from associated events.
- **_handle_isolate_host, _handle_block_network, _handle_lock_account**, etc.

Note: In the current software implementation, some extreme actions (like actually terminating a live Windows process via PID or isolating the host network adapter) are gracefully handled by logging a warning and noting that "admin confirmation is required" to ensure system safety during runtime, though the logical architecture supports full autonomous execution.